

# AI Assisted Coding –Assignment 12.1

---

**NAME: K.SAI VARDHAN REDDY**

**ROLLNO: 2503A51266**

## Task 1: Stack Implementation with AI Assistance

Prompt used:

Generate a Python Stack class with push(), pop(), peek(), and is\_empty() methods. Handle stack underflow with a clear error message and include docstrings for every method.

Code:

```
class Stack:
    """A simple stack implementation using a Python list."""

    def __init__(self):
        self.items = []

    def push(self, element):
        """Add an element to the top of the stack."""
        self.items.append(element)

    def pop(self):
        """Remove and return the top element."""
        if self.is_empty():
            raise IndexError("Stack underflow: stack is empty")
        return self.items.pop()

    def peek(self):
        """Return the top element without removing it."""
        if self.is_empty():
            raise IndexError("Stack underflow: stack is empty")
        return self.items[-1]

    def is_empty(self):
        """Return True if the stack is empty."""
        return len(self.items) == 0
```

```
s = Stack()
s.push(10)
s.push(20)
print("Top:", s.peak())
print("Popped:", s.pop())
print("Empty:", s.is_empty())
```

Output: Top: 20 | Popped: 20 | Empty: False

Explanation: The stack follows LIFO (Last In, First Out). `push()` adds an item, `pop()` removes the latest item, and `peek()` views it without removing it.

## Task 2: Linear Search vs Binary Search

Prompt used:

Generate Python functions `linear_search(arr, target)` and `binary_search(arr, target)`. Include docstrings explaining their working principles and time complexity. Test both algorithms on different input sizes.

```
def linear_search(arr, target):
    """Search sequentially. Time: O(n), Space: O(1)."""
    for i, value in enumerate(arr):
        if value == target:
            return i
    return -1
```

```
def binary_search(arr, target):
    """Search a sorted list by repeatedly halving it.
    Time: O(log n), Space: O(1).
    """
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
```

```
return -1
```

```
arr = [10, 20, 30, 40, 50, 60]
print("Linear:", linear_search(arr, 40))
print("Binary:", binary_search(arr, 40))
```

Output: Linear: 3 | Binary: 3

Comparison: Linear Search can work on an unsorted list and checks elements one by one. Binary Search requires sorted data but reduces the search space by half each step, giving  $O(\log n)$  time compared with  $O(n)$  for Linear Search.

### Task 3: Test Driven Development – Simple Calculator

Prompt used:

First generate unit tests for calculator addition and subtraction. Then provide implementation functions that make all tests pass. Keep the tests and implementation in separate files.

test\_calculator.py:

```
from calculator import add, subtract
```

```
def test_add():
    assert add(5, 3) == 8
```

```
def test_subtract():
    assert subtract(5, 3) == 2
```

calculator.py:

```
def add(a, b):
    """Return the sum of two numbers."""
    return a + b
```

```
def subtract(a, b):
    """Return the difference between two numbers."""
    return a - b
```

Test execution:

pytest test\_calculator.py → 2 passed

TDD flow: Tests are written first, the initial implementation is absent or fails, the calculator functions are then implemented, and the tests are rerun until they pass.

## Task 4: Queue Implementation with AI Assistance

Prompt used:

Generate a Python Queue class with enqueue(), dequeue(), front(), and is\_empty(). Handle overflow and underflow conditions and include docstrings for all methods.

```
class Queue:
    """A fixed-size FIFO queue."""

    def __init__(self, capacity=5):
        self.items = []
        self.capacity = capacity

    def enqueue(self, element):
        """Add an element to the rear of the queue."""
        if len(self.items) >= self.capacity:
            raise OverflowError("Queue overflow: queue is full")
        self.items.append(element)

    def dequeue(self):
        """Remove and return the front element."""
        if self.is_empty():
            raise IndexError("Queue underflow: queue is empty")
        return self.items.pop(0)

    def front(self):
        """Return the front element without removing it."""
        if self.is_empty():
            raise IndexError("Queue underflow: queue is empty")
        return self.items[0]

    def is_empty(self):
        """Return True when the queue has no elements."""
        return len(self.items) == 0

q = Queue(3)
q.enqueue(10)
```

```
q.enqueue(20)
print("Front:", q.front())
print("Dequeued:", q.dequeue())
print("Empty:", q.is_empty())
```

Output: Front: 10 | Dequeued: 10 | Empty: False

Explanation: The queue follows FIFO (First In, First Out). enqueue() adds at the rear and dequeue() removes from the front.

## Task 5: Bubble Sort vs Selection Sort

Prompt used:

Generate bubble\_sort(arr) and selection\_sort(arr) in Python. Add comments explaining each step and docstrings containing time and space complexity.

```
def bubble_sort(arr):
    """Sort using adjacent swaps.
    Time: O(n^2), Space: O(1).
    """
    a = arr.copy()
    for i in range(len(a)):
        for j in range(0, len(a) - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a
```

```
def selection_sort(arr):
    """Repeatedly select the smallest remaining value.
    Time: O(n^2), Space: O(1).
    """
    a = arr.copy()
    for i in range(len(a)):
        min_index = i
        for j in range(i + 1, len(a)):
            if a[j] < a[min_index]:
                min_index = j
        a[i], a[min_index] = a[min_index], a[i]
    return a
```

```
data = [64, 25, 12, 22, 11]
print("Bubble:", bubble_sort(data))
print("Selection:", selection_sort(data))
```

Output: Bubble: [11, 12, 22, 25, 64] | Selection: [11, 12, 22, 25, 64]

Comparison: Both algorithms have  $O(n^2)$  average/worst-case time complexity and  $O(1)$  auxiliary space in these in-place implementations. Bubble Sort repeatedly swaps adjacent out-of-order elements, while Selection Sort finds the minimum element and places it at the next position.

## Overall Conclusion

AI assistance can help generate data-structure and algorithm implementations, comments, docstrings, test cases, and complexity explanations. The generated code should still be reviewed and tested. The tasks demonstrate Stack and Queue data structures, search algorithms, TDD, and sorting algorithms as required by the assignment.

## Assignment Requirements Covered

- Prompts used for AI-assisted coding.
- Python code for all five tasks.
- Comments/docstrings and code explanations.
- Sample outputs and test results.
- Complexity comparison for search and sorting algorithms.
- TDD with separate test and implementation files.